Q1. Retail Order Processing Using Unity Catalog and Spark Declarative Pipelines

Difficulty: Easy — guided practical
Total marks: 20
Suggested duration: 25 minutes of working time, excluding compute startup and pipeline execution waiting time
Technology: Azure Databricks, Unity Catalog, PySpark, Spark Declarative Pipelines (SDP)

Problem statement

A retail company receives customer details and order transactions as two CSV files. The company needs a reusable pipeline that loads these files, joins each order with its customer, and produces a clean dataset containing completed orders and their calculated amounts.

Build a batch SDP pipeline with three materialized views:

1. `sdp1_yourname` — customer data.
2. `sdp2_yourname` — order data.
3. `sdp3_yourname` — joined and transformed completed orders.

After successfully running the pipeline, export the final dataset as CSV files to a Unity Catalog volume and validate the results using Databricks SQL.

Replace `yourname` consistently with your assigned identifier. For example, use `ak01`.

Prerequisites and assessment setup

The examiner must provide:

- A Unity Catalog-enabled Azure Databricks workspace.
- An existing catalog and schema with a configured managed storage location.
- Access to pipeline compute, notebook compute, and a SQL warehouse.
- Permission to create a volume and pipeline materialized views, and to read and write the assessment volume.
- Permission for the pipeline's run-as identity to read the input volume and create the required output objects.
- Both CSV files defined below.
- A starter notebook containing the imports and schema definitions, to support the suggested completion time.

Azure storage authentication and catalog creation must be completed before the assessment.

Source location clarification: `storagename224/raw` is a storage reference, not a complete ADLS URI. If ADLS is used, the examiner must supply the container name, full path, and configured access:

abfss://<container>@storagename224.dfs.core.windows.net/raw/

Alternatively, the examiner may provide the two CSV files directly for upload. Both options assess the same processing tasks.

Datasets

Use the following assessment datasets. These are newly defined sample files; they do not assume any existing contents in the ADLS location.

Dataset 1: customers.csv

Each row represents one customer. `customer_id` is unique.

customer_id,customer_name,city
C001,Ananya, chennai
C002,Bala,bengaluru
C003,Charan, hyderabad
C004,Divya,pune

Spaces before some city values are intentional.

Column	Required data type	Description
customer_id	STRING	Unique customer identifier and join key
customer_name	STRING	Customer name
city	STRING	City requiring removal of surrounding spaces and conversion to uppercase

Input row count: 4.

Dataset 2: orders.csv

Each row represents one order. `order_id` is unique.

order_id,customer_id,order_date,quantity,unit_price,order_status
O1001,C001,2026-10-01,2,500.00,COMPLETED
O1002,C002,2026-10-01,1,1200.00,COMPLETED
O1003,C003,2026-10-02,3,200.00,CANCELLED
O1004,C001,2026-10-02,1,750.00,COMPLETED

O1005,C004,2026-10-03,4,250.00,COMPLETED
O1006,C002,2026-10-03,2,300.00,PENDING

Column	Required input data type	Description
order_id	STRING	Unique order identifier
customer_id	STRING	Customer identifier used for the join
order_date	STRING	Date text in `yyyy-MM-dd` format
quantity	INT	Number of units ordered
unit_price	DECIMAL(10,2)	Price per unit
order_status	STRING	COMPLETED, CANCELLED, or PENDING

Input row count: 6.

All supplied customer references are valid. Deduplication, handling unmatched customers, and handling malformed records are outside this question.

Task 1: Prepare the Unity Catalog volume — 4 marks

Create a managed volume named:

<catalog>.<schema>.taskv_yourname

Create these directories inside the volume:

/Volumes/<catalog>/<schema>/taskv_yourname/input/
/Volumes/<catalog>/<schema>/taskv_yourname/output/

Upload or copy both source files into `input`, preserving their filenames:

input/customers.csv
input/orders.csv

Read each file separately. Do not read the entire input directory as one dataset because the files have different schemas.

Commands and utilities to use:

Command or utility	Purpose
`CREATE VOLUME IF NOT EXISTS`	Create the managed volume
`dbutils.fs.mkdirs()`	Create the input and output directories

Catalog Explorer upload, or an authorized file-copy operation	Place the supplied CSV files in input
`dbutils.fs.ls()`	Confirm both files are present

Completion requirement: The input directory contains both CSV files, and the output directory exists.

Task 2: Develop the SDP pipeline — 8 marks

Create a Python pipeline source file or notebook named:

SDP_Retail_Orders_yourname

Use these imports:

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
```

Use `@dp.materialized_view` for all three datasets because this assessment uses batch processing.

Task 2A: Load customer data — 2 marks

Create `sdp1_yourname` by reading `customers.csv` from the volume.

Requirements:

- Read the first row as column headers.
- Apply the explicit customer schema.
- Preserve all three columns and all four rows.
- Do not clean the city values in this first dataset.

Required APIs: `@dp.materialized_view`, `spark.read`, `.option("header", "true")`, `.schema()`, and `.csv()`.

Task 2B: Load order data — 2 marks

Create `sdp2_yourname` by reading `orders.csv` from the volume.

Requirements:

- Read the first row as column headers.
- Apply the explicit order schema.
- Preserve all six columns and all six rows.
- Keep `order_date` as STRING at this stage.
- Do not filter out any orders in this dataset.

Required APIs: `@dp.materialized_view`, `spark.read`, `.option("header", "true")`, `.schema()`, and `.csv()`.

Task 2C: Join and transform the data — 4 marks

Create `sdp3_yourname` by reading the two upstream materialized views through `spark.read.table()`.

Apply the following rules:

Operation	Required behavior	Functions or methods to use
Join	Perform an inner join on `customer_id`	`.join()`
Filter	Keep only rows where `order_status = 'COMPLETED'`	`.filter()` and `F.col()`
Standardize city	Remove leading and trailing spaces, then convert to uppercase	`F.trim()`, `F.upper()`, `.withColumn()`
Convert date	Convert `order_date` from STRING to DATE using `yyyy-MM-dd`	`F.to_date()`, `.withColumn()`
Calculate amount	Calculate `order_amount = quantity × unit_price`; store as DECIMAL(12,2)	`F.col()`, multiplication, `.cast()`, `.withColumn()`
Select output	Keep only the specified final columns	`.select()`

The final column order must be:

order_id
customer_id
customer_name
city
order_date
quantity
unit_price
order_amount

Return the resulting DataFrame from the decorated function.

Do not use `.write()`, `.save()`, `.collect()`, `.count()`, or `.display()` inside the three dataset-definition functions. Run validation actions in the separate validation notebook or SQL Editor.

Task 3: Validate, execute, and export — 6 marks

Task 3A: Configure and validate the pipeline — 2 marks

Create an SDP pipeline named:

SDP_Retail_Pipeline_yourname

Configure it to:

- Use the source file or notebook created in Task 2.
- Publish to the examiner-provided catalog and schema.
- Run as a triggered update rather than continuously.

Perform a **Dry run** to validate the pipeline definitions and dependencies. Resolve any reported errors before execution.

The dependency graph must show that `sdp3_yourname` depends on both `sdp1_yourname` and `sdp2_yourname`.

A successful dry run validates the pipeline definitions; it does not prove that the expected records have been processed.

Task 3B: Execute the pipeline — 2 marks

Run a full pipeline update and wait for successful completion.

Confirm that all three materialized views are available in the configured Unity Catalog schema:

<catalog>.<schema>.sdp1_yourname
<catalog>.<schema>.sdp2_yourname
<catalog>.<schema>.sdp3_yourname

Task 3C: Export the final result — 2 marks

After the pipeline succeeds, use a **separate notebook that is not included in the pipeline source**.

Read `sdp3_yourname` and export it as CSV to:

/Volumes/<catalog>/<schema>/taskv_yourname/output/completed_orders/

Use:

```
spark.read.table(...)
   .write
   .mode("overwrite")
   .option("header", "true")
   .csv(...)
```

Requirements:

- Export all eight final columns.
- Include column headers.
- Use overwrite mode so rerunning the export does not append duplicate records.
- Read the exported directory back and verify that it contains four data rows.

Spark may create multiple CSV part files and supporting files. A single CSV file is not required.

The CSV export is a file copy of the final dataset. The pipeline materialized views remain published under the configured catalog and schema.

Task 4: Validate the final output using SQL — 2 marks

In Databricks SQL Editor, query the three materialized views and confirm the following:

Validation	Expected result
Customer row count	4
Order row count	6
Final completed-order row count	4
Distinct order IDs in the final dataset	4
Total final order amount	3950.00

Display the final dataset ordered by `order_id`.

Expected final output

order_id	customer_id	customer_name	city	order_date	quantity	unit_price	order_amount
O1001	C001	Ananya	CHENNAI	2026-10-01	2	500.00	1000.00
O1002	C002	Bala	BENGALURU	2026-10-01	1	1200.00	1200.00
O1004	C001	Ananya	CHENNAI	2026-10-02	1	750.00	750.00
O1005	C004	Divya	PUNE	2026-10-03	4	250.00	1000.00

Also inspect the final schema and confirm:

- `order_date` is DATE.
- `order_amount` is DECIMAL(12,2).

Use `COUNT(*)`, `COUNT(DISTINCT order_id)`, `SUM(order_amount)`, `ORDER BY`, and `DESCRIBE TABLE` for the relevant checks.

Submission requirements

Submit:

1. The pipeline source file or exported source notebook.
2. The separate export and validation notebook, including its executed output.
3. The SQL validation queries and their results.
4. The pipeline link or pipeline ID.
5. The fully qualified volume name and output directory path.

Provide only these screenshots:

- **Screenshot 1 — Pipeline execution:** Successful actual update with the dependency graph showing all three datasets.
- **Screenshot 2 — Final SQL output:** The four completed orders, including all eight required columns.

Keep the dry-run result available in pipeline validation history or the available validation log for examiner review. Additional screenshots of volume creation, uploads, individual transformations, or intermediate configuration are not required.

Grading rubric

Task	Criterion	Marks
1	Correctly named managed volume	1
1	Input and output directories created	1
1	Both source files correctly placed and accessible	2
2A	Customer materialized view uses the correct file, schema, and headers; retains four rows	2
2B	Order materialized view uses the correct file, schema, and headers; retains six rows	2
2C	Inner join and completed-order filter	1
2C	City trimming and uppercase conversion	1
2C	Correct date conversion	1

2C	Correct amount calculation, decimal type, and final column selection	1
3A	Correct pipeline source, target configuration, and successful dry run	2
3B	Successful actual update producing all three materialized views	2
3C	Separate CSV export with headers, overwrite mode, and four-row read-back validation	2
4	SQL checks confirm counts, distinct order IDs, and total amount	1
4	Final records, column order, and required data types match the specification	1
	Total	**20**

Examiners must assess the submitted code, executed results, and pipeline history alongside the two screenshots. There are no additional screenshot requirements.